

# Topstep Combine Survival Engine

Complete project summary, experiment log, codebase state, major findings, current preset, and open investigation items

This document is a full handoff summary of the intraday MES Topstep Combine project. It captures the objective, architecture, key experiments, code changes, bug discoveries, validation framework, final mainline preset, and the allocator warmup bug that now defines the next versioned investigation.

**Date of summary:** March 4, 2026

**Primary instrument:** MES futures

**Primary optimization target:** maximize the probability of hitting +\$3,000 before ruin within 20 trading days under Topstep-style constraints.

**Current mainline preset:** `mainline_combine_v1`

# 1. Objective and governing constraints

The project is not an alpha-maximization exercise. It is a survival optimization problem. The system is designed to pass a Topstep \$50K Combine under a \$2,000 trailing drawdown, a \$1,000 daily loss limit, consistency constraints, and a hard 20-trading-day horizon.

The exact objective that guided the entire research effort is: **maximize the probability of hitting +\$3,000 before ruin within 20 trading days**. That framing matters, because many strategies that look acceptable on raw expectancy fail once sequence risk, volatility clustering, and drawdown boundaries are introduced.

The project therefore evolved into a stacked system with three layers: (1) edge engines, (2) a day-level allocator that routes the session to the appropriate engine, and (3) a sizing controller that tries to capture enough upside to pass while staying inside the combine limits.

## Initial engine design

- Core engine at the start: VWAP mean reversion (MR), fading deviations from VWAP using sigma-based thresholds.
- Entry elements: deviation threshold, reclaim logic, attempt limits per side, and regime filters intended to separate range from trend.
- Exit elements: VWAP target, hard stop, thesis-break MAE rule, optional time-stop, and several runner / staged-exit variants tested later.
- Validation framework: block-bootstrap Monte Carlo by session, stress scenarios, CI gates, and 20-day pilot packs.

## 2. Core codebase as understood from the work completed

The codebase grew around a validation-first research loop. The exact directory tree was not exhaustively dumped, but the following files and modules were directly referenced, edited, or extended during the project and form the active surface area of the system.

| File / module                                             | Role in the system                                                                                                  |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| strategies/mr_signal_engine.py                            | MR signal generation, reclaim modes, first-outside salvage, touch-latch logic, per-gate throughput ledger, regime   |
| simulation/mr_exit_simulator.py                           | Trade simulation and exit handling for MR and later ORB, including hold_bars, stop / target handling, partials      |
| replay_debug.py                                           | Replay execution path, signal creation, engine gating, ORB integration, runtime config snapshots, and artifact      |
| validation/validation_pack.py                             | Pack execution, runner orchestration, propagation of CLI flags, MC calls, artifact output, sizing / allocator integ |
| run_validation_pack.py                                    | Main CLI entrypoint. Later extended with preset loading, engine-mode flags, allocator flags, reclaim mode flag      |
| validation/trade_aggregator.py                            | Aggregate run metrics including avg_r, MAE percentiles, early-stop rates, drawdown-related summaries, and           |
| validation/reclaim_ablation.py                            | Reclaim on / off comparison helper.                                                                                 |
| validation/funnel_accounting.py                           | Daily and total drop-ledger reporting for bars, z-crosses, gate rejects, candidates, trades, and histograms.        |
| validation/throughput_ablation.py                         | D0-D5 grid runner for dedupe / attempt / cooldown throughput attribution.                                           |
| validation/day_allocator.py                               | Day-level allocator logic, first with ADX gate variants and later with hysteresis routing.                          |
| validation/sizing_policy.py                               | Fixed sizing, dynamic_v1, dynamic_v2, and dynamic_v3 sizing policies including trigger diagnostics.                 |
| validation/engine_mode_matrix.py                          | MR-only, ORB-only, and allocator-mixed comparison runs plus matrix artifact generation.                             |
| validation/sizing_comparison.py / run_sizing_100stress.py | 100-stress robustness sweeps for fixed and dynamic sizing policies.                                                 |
| replay_report.py                                          | Extended signal artifact schema, including ORB stop and target references and later diagnostic fields.              |
| indicators.py                                             | Contains the ADX calculator whose warmup behavior later exposed the allocator bug.                                  |
| regime_v1.py                                              | Referenced in the allocator audit; part of the regime decision path.                                                |
| config.py                                                 | Default parameters updated to the mainline preset values.                                                           |

| File / module                    | Role in the system                                                                                               |
|----------------------------------|------------------------------------------------------------------------------------------------------------------|
| presets/mainline_combine_v1.json | Frozen preset snapshot with allocator, ORB, sizing, account, MC, and timing parameters plus validated metrics    |
| visualize_forward_dashboard.py   | Forward-validation dashboard that reads artifacts and summarizes routing, headroom, sizing triggers, ORB quality |

The project also created a large artifact trail in `artifacts/validation_runs/...`, including pilot packs, trend-selected packs, matrix outputs, and JSON summaries used for later survival and robustness analysis.

### 3. Baseline MR findings before allocator and ORB took over

The first major discovery was that the MR engine had a real but capped edge. One early best configuration - known informally as **Exp 2** - used sigma 1.4, no time-stop, no runner, thesis-break plus hard stop, and produced roughly 7.25 trades/day, a 50% win rate, avg win around +0.86R, avg loss around -0.69R, and expectancy around +0.0919R. Drawdown p95 was around 717.

That looked acceptable at first, but the project then established a more important structural fact: **the right tail was thin**. Post-VWAP extension analysis showed that moves beyond VWAP + 0.8R were rare, beyond +1.2R were extremely rare, and beyond +1.4R effectively did not exist in the sample used. That killed the original runner thesis.

From there, multiple runner / staged-exit variants were tested. They generally reduced expectancy rather than increasing it. Removing the time-stop helped. Adding a runner did not manufacture convexity. The key conclusion was that the MR distribution capped near VWAP, so exit shaping could not solve the combine objective.

Monte Carlo then made the problem explicit: even with the best MR configuration, expected 20-day PnL was nowhere near the +\$3,000 target. The engine had edge, but not enough edge and not enough right-tail expression to pass the combine reliably on its own.

### 4. Reclaim ablation, soft-filter failure, and the real MR lesson

The next research branch attacked entry quality. The working hypothesis was that stacked filters might raise expectancy enough to make MR viable. Several single-variable and stacked experiments were run around reclaim, distance from VWAP, VWAP flatness, and reclaim strength.

Ablation results quickly showed that the new filters were not the real collapse driver. Candidate pools were already small, and the added filters only removed a few candidates each. That pushed the project toward a binary reclaim-mode test.

| Run            | candidate_count | trades/day | avg_r   | Win rate | Other notes                 |
|----------------|-----------------|------------|---------|----------|-----------------------------|
| R1 reclaim on  | 56              | 2.65       | -0.0504 | 41.5%    | MAE p95 23.5, dd_p95 301.7  |
| R0 reclaim off | 54              | 2.65       | +0.1270 | 47.2%    | MAE p95 26.35, dd_p95 294.3 |

This was one of the most important MR results. **Reclaim as an entry gate was net harmful in that sample**. It did not increase flow, but it reduced edge by filtering out winners or delaying entry. Reclaim-off became the correct MR baseline.

Several 'soft reclaim' variants were then tested. Soft-v1 and soft-v2 used directional body impulse and other veto ideas. They almost completely killed candidate flow. A range-impulse soft-v3 was also tested, but none of the variants improved the reclaim-off baseline. The decision became straightforward: keep MR as **cross-only, reclaim-off** and stop trying to rescue it through soft entry vetoes.

A 'first-outside salvage' rule was also tested. It increased candidates and trades modestly by handling sessions that opened already outside the threshold, but it diluted expectancy from 0.127 to about 0.093. Touch-latch mode was worse: it increased event flow slightly but collapsed avg\_r to roughly flat or negative. The clear MR conclusion was: **cross-only logic is quality-protective, reclaim is harmful, and touch-mode adds noise**.

### 5. Throughput audit and regime gating discovery

A full per-gate throughput ledger was then implemented. The D0-D5 grid toggled dedupe, attempt caps, and cooldown rules to determine where flow was being lost. This produced a clean answer.

| Variant              | z_cross | candidates | trades | trades/day | Main takeaway                       |
|----------------------|---------|------------|--------|------------|-------------------------------------|
| D0 baseline          | 54      | 54         | 43     | 2.15       | dedupe on                           |
| D1 dedupe off        | 54      | 54         | 53     | 2.65       | only meaningful throughput increase |
| D2 attempt off       | 54      | 54         | 43     | 2.15       | no effect                           |
| D3 cooldown off      | 54      | 54         | 43     | 2.15       | no effect                           |
| D4 / D5 interactions | 54      | 54         | 53     | 2.65       | matched dedupe-off behavior         |

The audit showed that **dedupe was a secondary translation suppressor**, but not the primary cause of low opportunity count. The real bottleneck remained the event definition itself. A separate test then turned the regime filter off entirely.

| Config                                  | z_cross | candidates | trades/day | avg_r   | dd_p95 |
|-----------------------------------------|---------|------------|------------|---------|--------|
| Reclaim off + first-outside + regime on | 54      | 59         | 2.90       | +0.0930 | 293.07 |
| Same with regime off                    | 94      | 111        | 4.80       | -0.0118 | 435.17 |

That result settled the MR debate. Turning regime off unlocked lots of extra flow, but the extra trades were bad trades. The regime filter was doing real quality work. The broader conclusion became: **MR is a range engine with a thin right tail, useful as a stabilizer but not as a growth engine.**

## 6. Initial ORB scaffold, false starts, and the trend-pack insight

Once MR was accepted as a stabilizer rather than the full solution, the research pivoted to a second engine. The chosen candidate was an opening-range breakout / continuation engine ('ORB') intended to provide convex tail behavior on trend days.

Initial ORB scaffolding was added end-to-end: signal path, stop / target references, simulator support, and allocator integration. However, early ORB runs often produced zero signals in mixed packs. This initially made ORB look non-viable.

A critical diagnostic run then changed the picture: a trend-selected 20-session pack showed that ORB was not structurally dead. In that trend pack, ORB\_ONLY produced 3 trades, all winners, with avg\_r around +0.6625. MR\_ONLY on the same trend sessions was disastrous, losing roughly \$2,505 with a 15.8% win rate and avg\_r around -0.47. That established a crucial architectural truth: **MR is destructive in trend and must be gated away; ORB can provide convexity on those days.**

Allocator tests on the trend pack then confirmed that routing away from MR was already valuable, even before ORB frequency was improved. Simple ADX-gated allocators and a hysteresis allocator dramatically improved results relative to MR-only on trend sessions.

| Mode on trend pack | Trades | MR trades | ORB trades | avg_r   | Total PnL |
|--------------------|--------|-----------|------------|---------|-----------|
| MR_ONLY            | 38     | 38        | 0          | -0.4696 | -\$2,505  |
| ORB_ONLY           | 3      | 0         | 3          | +0.6625 | +\$1,072  |
| ALLOC_V1_ADX20     | 4      | 1         | 3          | +0.47   | +\$1,064  |
| ALLOC_V2_HYST      | 7      | 4         | 3          | +0.19   | +\$958    |

## 7. Dollar-scaled survival and the first sizing results

The research then shifted from normalized R-space to Topstep-dollar survival modeling. This required feeding trade-level **pnl\_dollars \* contracts** into the Monte Carlo engine so that +\$3,000 hit rates and ruin probabilities had operational meaning.

Once scaled properly, a first set of comparisons showed that allocator-based routing was far superior to MR-only across pilot, random, and trend packs. A 100-draw robustness test found that fixed 2 contracts was often the best pure pass-probability lever, but it came with meaningfully higher ruin risk. Fixed 1 contract was safe but too slow.

| Allocator V2 - 100 draws | 1 contract | 2 contracts | 3 contracts |
|--------------------------|------------|-------------|-------------|
| Mean P_hit               | 0.1639     | 0.5799      | 0.7197      |
| Mean P_ruin              | 0.0052     | 0.0734      | 0.1582      |
| Mean P_daily_breach      | 0.0000     | 0.0000      | 0.0000      |
| Mean dd_p95              | \$886      | \$1,513     | \$1,814     |
| Mean equity_p50          | \$1,600    | \$2,404     | \$2,644     |
| Mean equity_p10          | \$536      | \$867       | \$625       |

The initial conclusion was that fixed 2c looked like the sweet spot. But later work exposed sampling and engine-expression issues, especially once the ORB path was more honestly measured. That eventually led to dynamic sizing research rather than a fixed-size decision.

## 8. ORB trigger broadening failed - until pullback\_v3 fixed the actual entry problem

An ORB expansion v2 matrix first tried to increase frequency by loosening the trigger. Four trigger modes were tested across multiple ADX filters on pilot\_20d: strict, break\_only, pullback\_only, and break\_or\_pullback.

Those results were poor. Strict mode had only modestly positive avg\_r with low frequency and a 40% win rate. break\_only and break\_or\_pullback roughly doubled trade count but turned expectancy negative and worsened the loss tail. pullback\_only fired zero trades. At first glance, that made ORB look like a low-frequency, mixed-condition engine with no safe way to broaden it.

Then a focused empirical debug on trend sessions changed everything. It found that **20/20 trend sessions had OR breaks**, there were 22 total breakouts, and **19 of those 22 had pullbacks**. Median pullback depth was 2.5 points, p75 depth about 5.4 points, and continuation after pullback was large - mean around 44 points. The issue was not ORB scarcity. The issue was that the trigger was standing in the wrong place.

This led to **ORB Pullback v3**, which explicitly detected a break, then waited up to a small number of bars for a pullback back to the OR level within a configurable point tolerance, and entered on touch-only recovery. The resulting 12-cell matrix on 20 trend sessions was the breakthrough moment of the project.

| Config                     | Trades | Trades/session | WR     | avg_r   | p10_r | p90_r | PnL     |
|----------------------------|--------|----------------|--------|---------|-------|-------|---------|
| Old break baseline         | 3      | 0.15           | 100.0% | +0.6625 | 0.18  | 1.11  | \$1,072 |
| pullback_v3 tol=5.0 bars=3 | 33     | 1.65           | 84.8%  | +0.6906 | -0.70 | 1.21  | \$4,460 |
| pullback_v3 tol=5.0 bars=5 | 32     | 1.60           | 84.4%  | +0.6747 | -0.73 | 1.21  | \$4,283 |
| pullback_v3 tol=6.5 bars=3 | 34     | 1.70           | 82.4%  | +0.6557 | -0.67 | 1.20  | \$4,442 |

The chosen ORB default became: **pullback\_v3, tolerance 5.0 points, max\_bars 3, entry mode touch\_only**. That preserved convexity while expanding opportunity by roughly an order of magnitude versus the old break-only logic.

## 9. Full-system evaluation after ORB Pullback v3

With ORB fixed, the allocator system was re-evaluated end to end on several packs. This was the point where the intended architecture finally became visible in the data.

| Pack    | Arm     | Trades | T/D  | WR    | avg_r   | dd_p95 | eq_p50 |
|---------|---------|--------|------|-------|---------|--------|--------|
| trend20 | MR_ONLY | 38     | 1.90 | 15.8% | -0.4696 | \$579  | -\$396 |

| Pack      | Arm           | Trades | T/D  | WR    | avg_r   | dd_p95 | eq_p50 |
|-----------|---------------|--------|------|-------|---------|--------|--------|
| trend20   | ORB_ONLY      | 33     | 1.65 | 84.8% | +0.6906 | \$80   | \$512  |
| trend20   | ALLOC_V2_HYST | 33     | 1.65 | 81.8% | +0.6571 | \$70   | \$485  |
| pilot_20d | MR_ONLY       | 53     | 2.65 | 47.2% | +0.1270 | \$294  | \$141  |
| pilot_20d | ORB_ONLY      | 30     | 1.50 | 66.7% | +0.1939 | \$177  | \$137  |
| pilot_20d | ALLOC_V2_HYST | 50     | 2.50 | 72.0% | +0.4924 | \$84   | \$542  |
| random02  | MR_ONLY       | 51     | 2.55 | 37.3% | -0.0273 | \$475  | -\$39  |
| random02  | ALLOC_V2_HYST | 47     | 2.35 | 72.3% | +0.6201 | \$99   | \$688  |

At this point, the project had a coherent architecture: **MR for range, ORB pullback\_v3 for trend, allocator v2 hysteresis to route the day, and a sizing layer above that.**

## 10. Dynamic sizing evolution and the final mainline policy

The first dynamic sizing policies (v1 and v2) were too conservative. In robustness tests, they often behaved identically to fixed 1c because the earned-upsize conditions almost never fired. Once ORB pullback\_v3 improved trend-day expression, sizing calibration became meaningful.

Dynamic v3 was then introduced with multiple activation triggers: traction, first-trade-win, and an ORB-day auto-upsize path, plus giveback-based downshifts, headroom checks, and a profit lock. A calibration sweep across 100 draws found that **dynamic\_v3\_75\_25** was the best of the initial v3 family. After plumbing fixes, the ORB auto-upsize path was tested honestly and beat the base v3 arm.

| 200-draw final confirmation | fixed_1c | fixed_2c | dynamic_v3_orb_start |
|-----------------------------|----------|----------|----------------------|
| P_hit                       | 0.158    | 0.561    | 0.391                |
| P_ruin                      | 0.0124   | 0.1005   | 0.0185               |
| P_daily                     | 0.000    | 0.077    | 0.000                |
| Eq_p50                      | \$1,552  | \$2,299  | \$2,216              |
| Eq_p10                      | \$327    | \$343    | \$680                |
| dd_p95                      | \$1,182  | \$1,798  | \$1,327              |

This was the final sizing decision. **dynamic\_v3\_orb\_start** captured roughly 96% of fixed 2c median equity while carrying only about 18% of fixed 2c ruin risk. It also eliminated daily-loss breach risk in that confirmation run. That is why it became the recommended mainline sizing policy.

## 11. Frozen preset and forward-validation infrastructure

The project then froze the current best stack as **presets/mainline\_combine\_v1.json**. Defaults in config.py were updated to match the preset. A **--preset** flag was added to the CLI. A reproducibility check with a new seed showed metrics within normal variance, with the alternate seed actually slightly stronger.

Forward-validation tooling was also improved. The run console and artifact summaries were expanded to show v3-specific sizing diagnostics. A dashboard script (**visualize\_forward\_dashboard.py**) was created to summarize preset integrity, daily PnL, 2c activation, allocator distribution, headroom use, and ORB pullback quality.

Forward replay on **pilot\_20d** and then **extended\_60d** showed that the stack was profitable and risk-safe. However, a new tension emerged: the allocator never routed any session to ORB in live-style replay, even though the research story had validated ORB routing earlier.

## 12. Live allocator audit: the critical bug discovered after forward validation

A formal live audit of the allocator exposed the root cause. The issue was not threshold placement. It was **indicator warmup mismatch**.

The ADX calculator used in the allocator requires exactly 28 bars before producing a non-zero ADX value when period = 14. The live replay allocator made its regime decision in the 09:30-09:45 ET open window, where ADX is guaranteed to be 0.0. Therefore, `allocator_open_window_adx` was always empty, and the hysteresis allocator always defaulted to MR.

The research calibration runner, by contrast, used the first 12 **non-zero** ADX values from the features snapshot, with no open-window restriction. That effectively meant it was using bars from around 11:45 ET onward - over two hours after the open. In other words, the research allocator and the live allocator were not making the same decision at all.

| Allocator live audit result                       | Value                           |
|---------------------------------------------------|---------------------------------|
| Total sessions audited                            | 58                              |
| Actual ORB-routed sessions in live replay         | 0 (0%)                          |
| Would-be ORB sessions if warm ADX were used later | 31 (53%)                        |
| First non-zero ADX bar                            | 28 for every session            |
| Conclusion                                        | Warmup bug, not threshold issue |

This is the current frontier. **mainline\_combine\_v1 remains frozen and profitable**, but its live allocator implementation does not reflect the research route logic that validated ORB. Any fix must be versioned. The recommended path is not to lower thresholds, but to replace open-session ADX gating with a signal that is actually available by 09:45 ET.

### Proposed v1\_1 investigation path

- Keep `mainline_combine_v1` untouched.
- Create a versioned investigation branch / preset for an allocator open fix.
- Do not use ADX-14 at the open for ORB routing.
- Replace the open-session regime decision with a proxy available by 09:45 ET, such as opening-range width relative to ATR, early directional impulse, and OR-break persistence.
- Compare current `ALLOC_V2_HYST` against a new `open_proxy_v1` allocator on `pilot_20d`, `extended_60d`, and `trend20`.
- Only consider `mainline_v1_1` if the new allocator routes some real sessions to ORB, preserves or improves PnL, and does not damage headroom safety.

## 13. Current status: what is settled, what is open

| Settled / validated                                                                                          | Open / under investigation                                                                                           |
|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| MR engine: cross-only, reclaim-off, regime-gated. Useful as a stabilizer.                                    | ADX on <code>extended_60d</code> as a proxy for regime. New warmup bug makes the live hysteresis allocator unusable. |
| ORB engine: <code>pullback_v3</code> with tolerance 5.0, <code>max_bars</code> 3, <code>touch_only</code> 1. | Need strong trendline candidate and materialized proxy available before open.                                        |
| Sizing: <code>dynamic_v3_orb_start</code> is the best risk-adjusted sizing policy tested so far.             | Need offline comparison of <code>open_proxy</code> allocator variants before any live mainline change.               |
| Preset + dashboard + version discipline are in place.                                                        | Need forward validation of any allocator-open fix under explicit version control.                                    |

## 14. Recommended immediate next steps

- Keep running `mainline_combine_v1` as the frozen profitable baseline for controlled forward validation.
- Create a documented allocator-open-fix investigation branch (for example, `mainline_combine_v1_1_allocator_openfix`).

- Implement an open\_proxy allocator based on opening-range width / ATR, early directional impulse, and breakout persistence.
- Run side-by-side comparisons versus ALLOC\_V2\_HYST on pilot\_20d, extended\_60d, and trend20.
- Do not silently adjust thresholds or mutate the mainline preset in place.
- Preserve version discipline for every future change.

## 15. Final one-paragraph handoff

The project began as an attempt to force a VWAP mean-reversion system to pass a Topstep Combine, but the research made clear that MR alone could not do it. The solution became a three-layer system: MR for range, ORB for trend, allocator for day routing, and a sizing controller on top. ORB looked sparse until the pullback\_v3 trigger revealed that trend sessions were full of missed opportunities; dynamic sizing looked inert until ORB expression improved enough to make 2c activation worthwhile. The resulting mainline stack - allocator v2 hysteresis, ORB pullback\_v3, and dynamic\_v3\_orb\_start sizing - is profitable, robust, and version-frozen. The only unresolved issue is that the live allocator uses ADX-14 in an unwarmed open window and therefore cannot actually express ORB in forward replay. That bug is now known precisely. The system is at the point where further progress depends less on alpha discovery and more on a clean, versioned allocator-open fix.

End of summary.